

A DISTRIBUTED AND SCALABLE ORDER FULFILLMENT ARCHITECTURE FOR E-COMMERCE PLATFORMS

Ronald Judin¹ Kalju Jake Nekvasil¹

¹Institute of Computer Science, University of Tartu

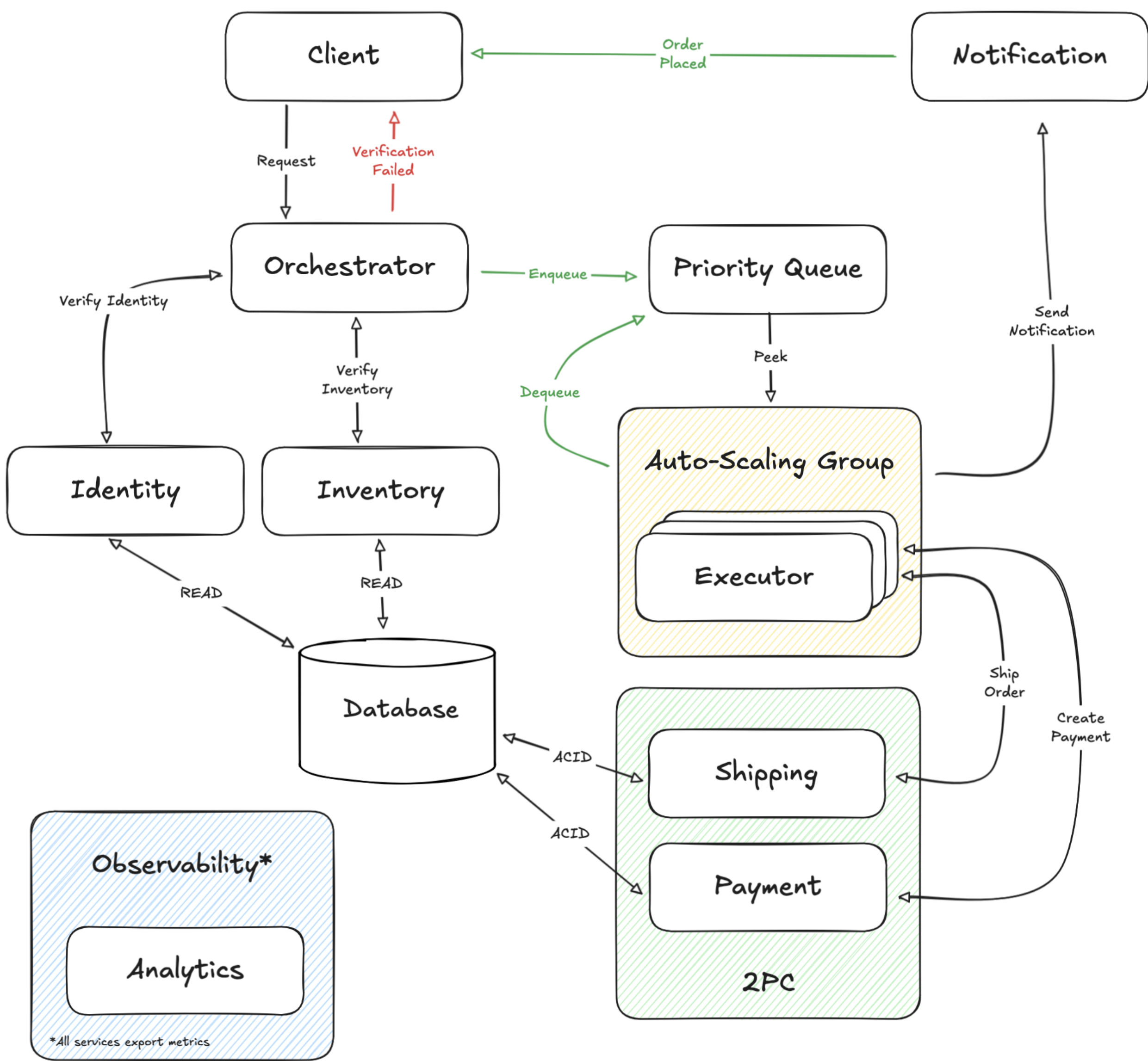


Introduction

We propose a distributed order fulfillment architecture designed for high-throughput e-commerce platforms. Modern online retailers like Amazon need reliability during traffic spikes and consistency for concurrent orders. Our system addresses these with orchestrated parallel verification, priority-based task queuing, and fault-tolerant two-phase commits for payments and shipping.

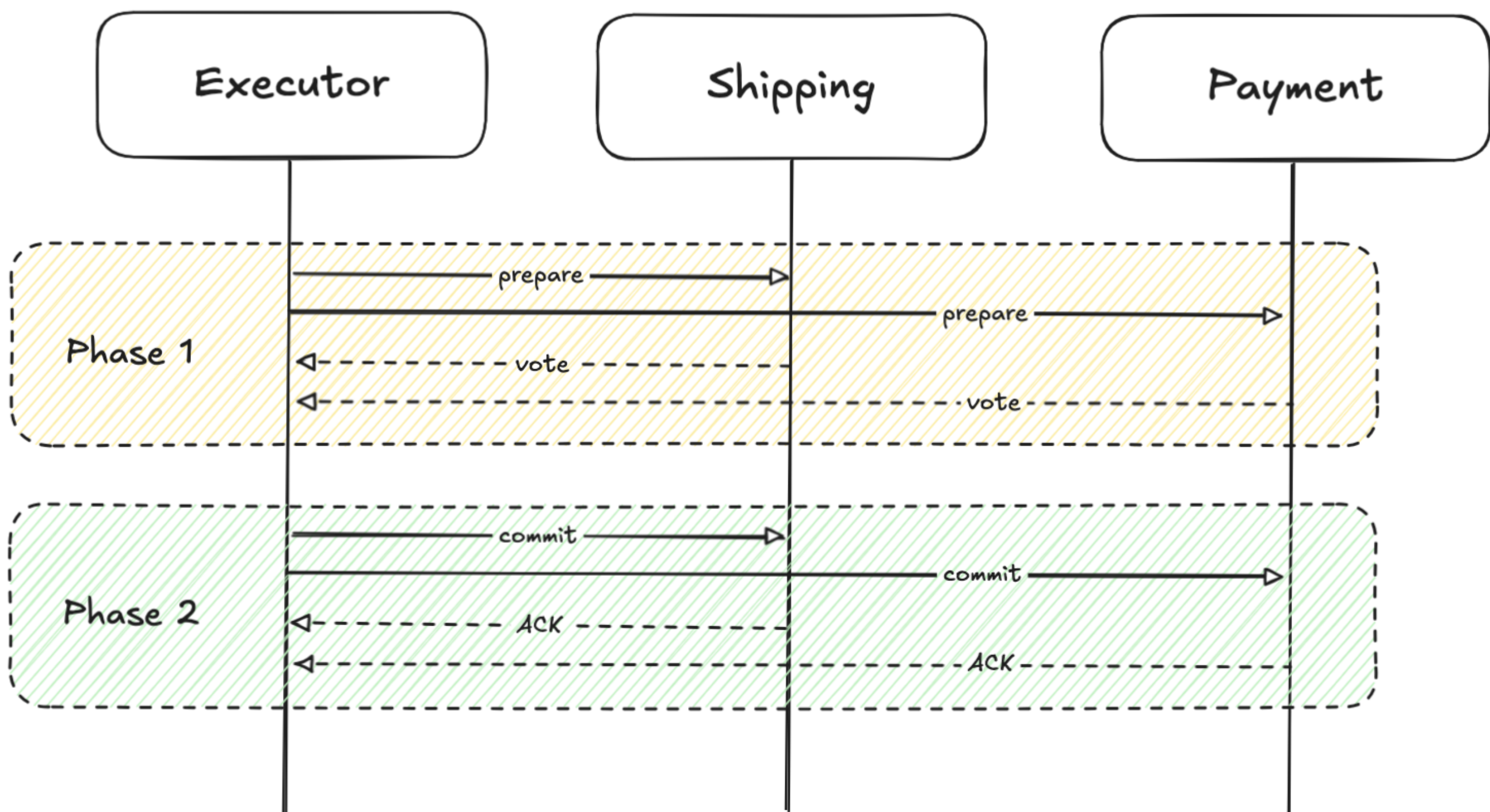
System Model

The system follows a **distributed orchestration** model. A client submits an order to the **orchestrator**, which initiates **multithreaded parallel calls** to the **identity** and **inventory services**. These services perform **READ requests** against the **leader database** to validate the request. After receiving **APPROVED** responses from both services, the orchestrator adds the order to a **priority queue**. **Auto-scaling executors** then process orders from the queue, coordinating **two-phase commit (2PC)** transactions with the **shipping** and **payment services** which guarantees **ACID compliance** for writes. After both services acknowledge (**ACK**) the **Commit phase**, the executor **dequeues** the order and triggers the **notification service** to inform the user. Throughout this workflow, the **analytics service** asynchronously logs requests, errors, and metrics for observability. The design prioritizes **scalability** (with auto-scaling and multithreading), **consistency** (with 2PC and leader DB reads), and **fault tolerance** (with queue-based retries and analytics monitoring).



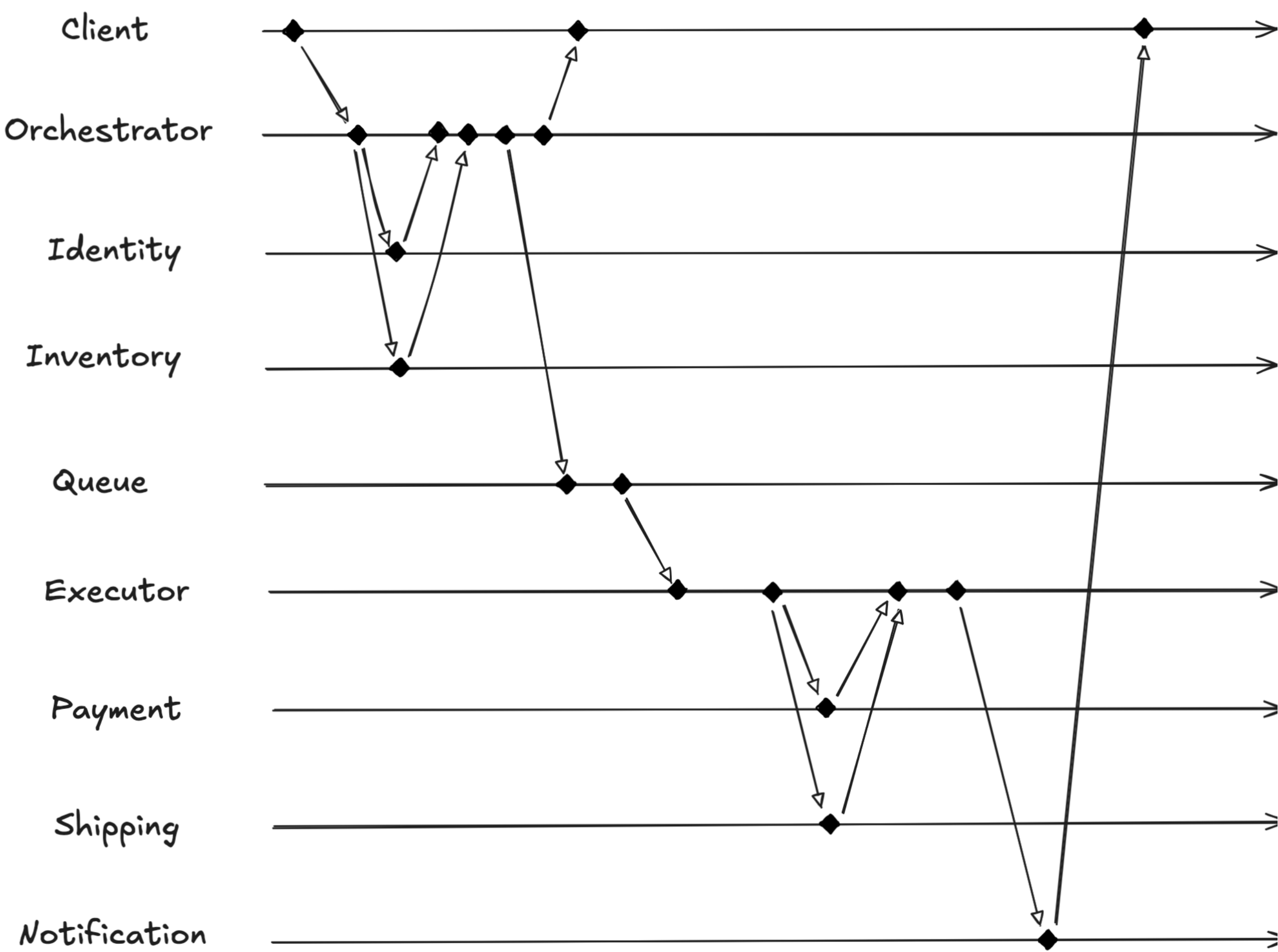
Commitment Protocol

The system uses **two-phase commit (2PC)** between the **executor**, **payment**, and **shipping service** to ensure atomic order processing. To handle failures, the system uses **optimistic exponential backoff** retries. If retries breach a set threshold, the **analytics service** can be used to track all logs, errors, and other specific order failure information.



Vector Clocks

The **analytics service** uses vector clocks to track causal relationships between events across the entire workflow from order request to placement. This tracking can be used for monitoring **race conditions** in real-time and debugging in the future. Below shows the general expected vector clock tracking timeline. For readability purposes, we do not display the vector value at every event.



Replication

Two types of replication are used in our distributed system:

Executor Replication (Queue-Based)

Auto-scaling executors are stateless and rely on the **priority queue** for order assignment. Unlike database replication:

- **Durability**: Orders persist in the queue until successfully processed.
- **Elasticity**: Executors scale dynamically based on queue size.

Database Replication (Leader-Follower)

The database system uses **synchronous leader-follower replication**. The **leader** handles all writes and the **followers** serve read requests. This behavior allows for:

- **Strong consistency**: Followers mirror leader state before acknowledging reads.
- **Fault tolerance**: Automatic elections during leader replica failure minimizes database downtime.

